Machine Learning 1 — Homework 2

Fall 2026 | DATA 37711

Jiahao Cheng | 12549801 | Code Repo: homework_2

Problem 1

(a)

Let $g(x) = (g_1(x), g_2(x))$. I define the link function

$$\ell : \mathbb{R}^2 \to \mathbb{R} \times (0, \infty), \qquad \ell(a, b) = (a, e^b).$$

Then

$$(f_\mu(x), f_\sigma(x)) = \ell(g(x)) = (g_1(x), e^{g_2(x)}).$$

This leaves the conditional mean unrestricted and ensures that the conditional standard deviation is strictly positive.

(b)

Under the assumed conditional Gaussian model, the density is

$$p(y_i \mid x_i) = \frac{1}{\sqrt{2\pi} f_\sigma(x_i)} \exp\left(-\frac{(y_i - f_\mu(x_i))^2}{2 f_\sigma(x_i)^2} \right).$$

I minimize the average negative log-likelihood:

$$L = \frac{1}{n} \sum_{i=1}^{n} \left[\log f_\sigma(x_i) + \frac{(y_i - f_\mu(x_i))^2}{2 f_\sigma(x_i)^2} \right],$$

where the constant $\frac{1}{2}\log(2\pi)$ has been omitted.

Using the link function from part (a), this becomes

$$L = \frac{1}{n} \sum_{i=1}^{n} \left[g_2(x_i) + \frac{1}{2}(y_i - g_1(x_i))^2 e^{-2 g_2(x_i)} \right].$$

(c)

I split the 523,554 observations into training, validation, and test sets containing 366,487, 78,533, and 78,534 observations, using a random seed of 42.

I considered two baselines: a Gaussian model with a constant mean and standard deviation estimated from the training set, and a linear regression model with a constant standard deviation estimated from its training residuals.

The MLP had architecture $7 \to 64 \to 64 \to 2$, with ReLU activations after the hidden layers. I minimized the Gaussian negative log-likelihood using Adam with learning rate 10^{-3} and batch size 256. Training used a maximum of 200 epochs and early stopping with patience 15. The checkpoint with the lowest validation NLL was selected, which occurred at epoch 109.

Model	Validation NLL	Test NLL	Test RMSE (min)
Constant Gaussian	2.5471	2.5486	7.7569
Linear Gaussian	2.5180	2.5208	7.5445
Gaussian MLP	1.6919	1.8553	3.9491

All reported NLL values omit the constant $\frac{1}{2}\log(2\pi)$.

The MLP outperformed both baselines on test NLL and RMSE. Compared with the linear baseline, it reduced test RMSE by approximately 47.7%.

(d)

Under the fitted model,

$$\begin{aligned} P(Y < 45 \mid X = x) &= P\left(\frac{Y - f_\mu(x)}{f_\sigma(x)} < \frac{45 - f_\mu(x)}{f_\sigma(x)} \;\middle|\; X = x\right) \\ &= \Phi\left(\frac{45 - f_\mu(x)}{f_\sigma(x)}\right), \end{aligned}$$

where Φ is the standard normal cumulative distribution function. Using the link function from part (a), the predicted probability is

$$\hat{p}(x) = \Phi\left((45 - g_1(x))e^{-g_2(x)}\right).$$

(e)

For each test observation, I computed

$$\hat{p}_i = \Phi\left(\frac{45 - f_\mu(x_i)}{f_\sigma(x_i)}\right), \qquad z_i = \mathbf{1}\{y_i < 45\}.$$

I evaluated the predicted probabilities using the Brier score,

$$\text{Brier} = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} (\hat{p}_i - z_i)^2,$$

and classification accuracy using $\hat{z}_i = \mathbf{1}\{\hat{p}_i \geq 0.5\}$.

Model	Brier score	Accuracy
Constant Gaussian	0.004457	99.5543%
Linear Gaussian	0.004455	99.5543%
Gaussian MLP	0.002782	99.6817%

The MLP achieved the lowest Brier score and highest accuracy. However, 99.5543% of test trips lasted less than 45 minutes, so predicting this outcome for every observation already gives high accuracy.

Problem 2

(a)

The dataset contains 397,924 observations. The cardinalities of `StockCode` and `CustomerID` are 3,665 and 4,339.

Full one-hot encoding would produce 8,004 features. A dense representation would be memory-intensive, but sparse one-hot encoding is feasible because each observation has only two nonzero entries. Target encoding and embeddings provide lower-dimensional alternatives.

I split the data into training, validation, and test sets using proportions of 70%, 15%, and 15%, respectively, with random seed 42.

(b)

I used `category_encoders.TargetEncoder` to encode StockCode and CustomerID using smoothed means of Quantity computed from the training set. The fitted encoder was then applied to the test features.

Since Quantity is heavy-tailed, I fitted a linear regression model using squared-error loss on $\log(1 + \text{Quantity})$:

$$L(\beta_0, \beta) = \frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} \left[\log(1 + y_i) - (\beta_0 + \beta^\top z_i)\right]^2,$$

where z_i contains the two target-encoded features.

The model achieved a test log-scale RMSE of 0.898193.

(c)

I used a separate trainable 16-dimensional embedding for StockCode and CustomerID. The embeddings were concatenated and passed through a hidden layer with 32 units and ReLU activation, followed by a scalar output layer. Category vocabularies were constructed from training data, and unseen categories were mapped to a fixed zero embedding.

The model minimized squared-error loss on the log target:

$$L(\theta) = \frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} \left[\log(1 + y_i) - f_\theta(s_i, c_i)\right]^2,$$

where s_i and c_i are the product and customer indices.

I trained with Adam, learning rate 10^{-3}, and batch size 1024. Early stopping used validation loss with patience 5, selecting the checkpoint at epoch 60.

The test log-scale RMSE was 0.537094, compared with 0.898193 for target encoding with linear regression, a reduction of approximately 40.2%. Thus, the embedding neural network performed better in this experiment.

(d)

I concatenated the two 16-dimensional embeddings with the two target-encoded features, producing a 34-dimensional input. The target-encoded features were standardized using training-set statistics. The combined input was passed through a hidden layer with 32 units and ReLU activation, followed by a scalar output layer.

The model minimized squared-error loss on $\log(1 + \text{Quantity})$. I used the same optimizer, learning rate, batch size, and early-stopping rule as in part (c). The best checkpoint occurred at epoch 70.

Model	Test log RMSE
Target encoding + LR	0.898193
Embeddings	0.537094
Target encoding + embeddings	0.541630

The combined model outperformed target encoding with linear regression, but performed slightly worse than the embedding-only model.

(e)

An embedding layer assigns each product or customer a trainable vector. These vectors are learned jointly with the prediction network, allowing the model to represent multiple aspects of category-specific purchasing patterns. The subsequent nonlinear layers can also model interactions between product and customer representations.

This is useful because both features have high cardinality. Unlike target encoding, which summarizes each category with a single average quantity, embeddings provide a multidimensional representation.

On the test set, the embedding model achieved a log RMSE of 0.537094, compared with 0.898193 for target encoding with linear regression.

The combined model achieved a slightly higher test log RMSE of 0.541630. The target-encoded features may offer limited additional information beyond the embeddings.

Problem 3

(a)

I visualized the training data using a 3D scatter plot, with x_1 and x_2 on the horizontal axes and the response y on the vertical axis.

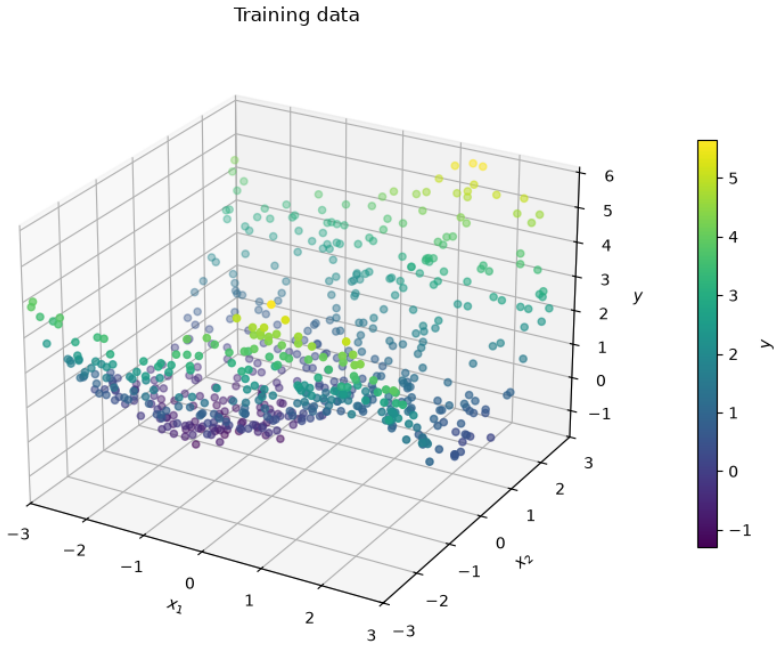


Figure 1: Training observations, with color indicating y.

(b)

I constructed paired sine and cosine features using fixed random projections and fitted ridge regression on these features. The intercept was not penalized.

I varied $h \in \{10, 15, 20, \ldots, 70\}$, corresponding to $D = 2h$ random features, while fixing $\sigma = 1$ and $\lambda = 10^{-3}$.

The selected configuration was $h = 15$ ($D = 30$), with validation MSE 0.035899.

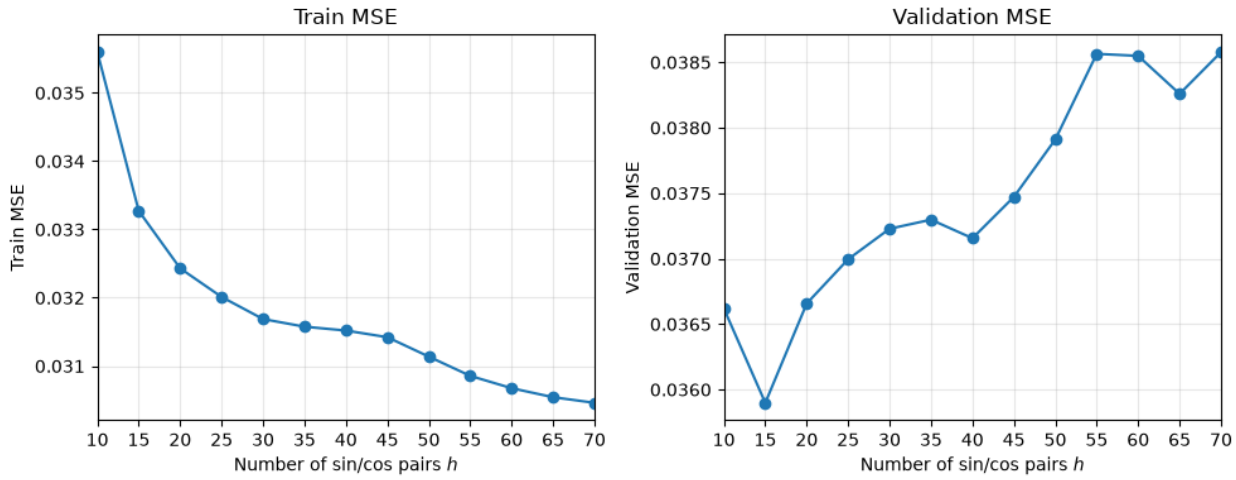


Figure 2: Training and validation MSE as functions of h.

(c)

I implemented a one-hidden-layer MLP with architecture $2 \to H \to 1$, using ReLU activation in the hidden layer. The model minimized training MSE using Adam with learning rate 10^{-3}, batch size 128, and 1000 training epochs.

I varied the hidden dimension over

$$H \in \{16, 32, 64, 128, 256, 512, 1024, 2048\}.$$

The lowest validation MSE was achieved at $H = 512$, with training MSE 0.027212 and validation MSE 0.039189.

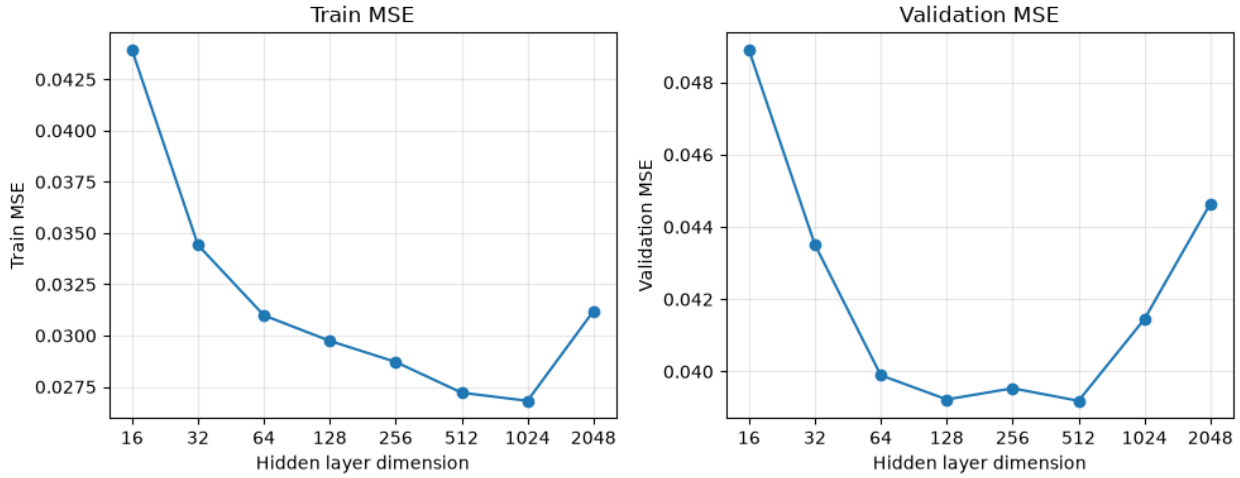


Figure 3: Training and validation MSE as functions of the MLP hidden dimension.

(d)

I refitted both models on the training set using their selected configurations: $D = 30$ for RKS and hidden dimension $H = 512$ for the MLP. RKS achieved a slightly lower validation MSE.

The smooth sinusoidal features and ridge regularization may fit the smooth structure of this dataset, which could explain the slightly lower validation error of RKS.

Model	Validation MSE
RKS	0.035899
One-hidden-layer MLP	0.039189

Both fitted surfaces reproduce the main patterns visible in the training data from part (a). They show a valley near $(x_1, x_2) = (-1.5, 0)$, increasing responses as x_2 moves toward either boundary, and a rise along x_1 toward approximately $x_1 = 1.5$. Thus, both models capture the broad valley and elevated regions in the observations. The two fitted surfaces are visually similar.

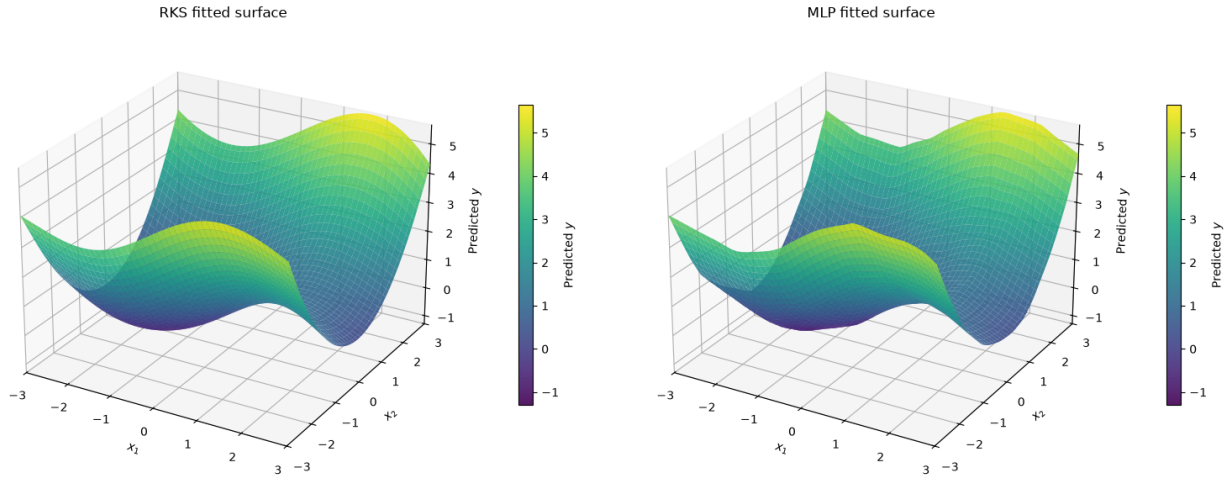


Figure 4: Fitted RKS function (left) and MLP function (right).